



Universidade de Brasília
Departamento de Ciência da Computação

Redes de Computadores Projeto Crawler

Gabriel Menezes de Araújo
Matrícula: 241020803

1 INTRODUÇÃO

Este relatório descreve a implementação de um web crawler desenvolvido como trabalho prático da disciplina de Redes de Computadores. O objetivo principal foi consolidar o entendimento prático do protocolo HTTP por meio da implementação manual de um programa capaz de navegar recursivamente por páginas web, sem utilizar bibliotecas prontas de alto nível para comunicação HTTP.

O crawler desenvolvido é capaz de:

- Estabelecer conexões TCP com servidores HTTP
- Construir e enviar requisições HTTP manualmente
- Interpretar respostas HTTP, incluindo cabeçalhos e códigos de status
- Extrair links de páginas HTML
- Navegar recursivamente por todas as páginas acessíveis
- Tratar redirecionamentos (códigos 301 e 302)
- Gerar um relatório detalhado de todas as requisições realizadas

O trabalho foi implementado em Python 3, utilizando exclusivamente a biblioteca **socket** para comunicação de rede, e **BeautifulSoup** apenas para auxiliar na extração de links do HTML.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Protocolo HTTP

O Hypertext Transfer Protocol (HTTP) é um protocolo da camada de aplicação utilizado para transferência de documentos hipermídia. Ele opera sobre o protocolo TCP e segue o modelo cliente-servidor, onde:

- O cliente (neste caso, o crawler) envia requisições ao servidor
- O servidor processa a requisição e envia uma resposta

2.2 Estrutura de uma Requisição HTTP

Uma requisição HTTP comum possui a seguinte estrutura:

```
GET /caminho/recurso HTTP/1.1
Host: exemplo.com
User-Agent: MeuCrawler/1.0
Connection: close
```

Os principais componentes são:

- **Linha de requisição:** contém o método (GET), o caminho do recurso e a versão do HTTP
- **Cabeçalhos:** pares chave-valor que fornecem informações adicionais
- **Linha em branco:** separa cabeçalhos do corpo (quando há)

2.3 Estrutura de uma Resposta HTTP

Uma resposta HTTP possui a seguinte estrutura:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<html>...conteúdo...</html>
```

Os componentes principais são:

- **Linha de status:** contém a versão do HTTP, o código de status e uma mensagem descritiva
- **Cabeçalhos:** metadados sobre a resposta (tipo de conteúdo, tamanho, etc.)
- **Corpo:** conteúdo da resposta (HTML, imagem, texto, etc.)

2.4 Códigos de Status HTTP

Os códigos de status indicam o resultado da requisição. Os mais relevantes para este trabalho são apresentados na Tabela 1:

Código	Significado	Ação tomada pelo crawler
200	OK - Requisição bem sucedida	Extrair links do corpo HTML
301	Moved Permanently	Seguir o redirecionamento (header Location)
302	Found	Seguir o redirecionamento (header Location)
404	Not Found	Registrar erro, não extrair links
500	Internal Server Error	Registrar erro, não extrair links

Tabela 1: Códigos de status HTTP tratados pelo crawler

2.5 Sockets TCP

Um **socket** é um endpoint de comunicação que permite a troca de dados entre processos, possivelmente em diferentes máquinas. No contexto deste trabalho, o crawler cria um socket TCP que:

1. Conecta-se ao servidor na porta 80 (porta padrão do HTTP)
2. Envia a requisição HTTP como uma sequência de bytes
3. Recebe a resposta do servidor, também como bytes
4. Fecha a conexão

3 FERRAMENTAS UTILIZADAS

3.1 Linguagem de Programação

Python 3 foi escolhido pelas seguintes razões:

- Sintaxe clara e expressiva
- Biblioteca padrão rica, incluindo **socket** para comunicação de baixo nível
- Facilidade de manipulação de strings para construção de requisições
- Suporte nativo a estruturas de dados como **set** e **dict**

3.2 Bibliotecas Utilizadas

A Tabela 2 resume as bibliotecas utilizadas e suas respectivas justificativas:

Biblioteca	Uso	Justificativa
<code>socket</code>	Comunicação TCP com servidores	Única forma de implementar HTTP manualmente
<code>urllib.parse</code>	Parsing de URLs (<code>urlparse</code> , <code>urljoin</code>)	Auxilia no tratamento de URLs
BeautifulSoup (<code>bs4</code>)	Parsing de HTML e extração de links	Permitido pela especificação, apenas para parsing
<code>datetime</code>	Timestamp do relatório	Para registrar data/hora da execução

Tabela 2: Bibliotecas utilizadas no projeto

3.3 Bibliotecas NÃO Utilizadas

Conforme exigência do trabalho, as seguintes bibliotecas foram **evitadas** por abstraírem o protocolo HTTP:

- `requests`
- `urllib.request`
- `httpx`
- `http.client`

4 DECISÕES DE IMPLEMENTAÇÃO

4.1 Estrutura Geral do Crawler

O crawler foi implementado como uma classe `WebCrawler` para organizar o código. A classe contém os seguintes atributos:

```
class WebCrawler:
    def __init__(self):
        self.visitados = set()    # URLs já processadas (evita loops)
        self.fila = []            # URLs aguardando processamento
        self.relatorio = []       # Resultados para o relatório final
```

4.2 Tratamento de URLs (`parse_url`)

A função `parse_url` utiliza `urlparse` para decompor uma URL em seus componentes:

```
def parse_url(self, url):
    parsed = urlparse(url)
    host = parsed.hostname or 'localhost'
    porta = parsed.port or 80
    caminho = parsed.path or '/'
    if parsed.query:
        caminho += '?' + parsed.query
    return host, porta, caminho
```

Decisão importante: A porta padrão é 80, mas o código suporta portas diferentes especificadas na URL.

4.3 Construção Manual da Requisição HTTP

Esta é a parte central do trabalho. A função `construir_requisicao` monta a requisição HTTP manualmente:

```
def construir_requisicao(self, host, caminho):
    request = f"GET {caminho} HTTP/1.1\r\n"
    request += f"Host: {host}\r\n"
    request += "User-Agent: MeuCrawler/1.0\r\n"
    request += "Connection: close\r\n"
    request += "\r\n"
    return request.encode('utf-8')
```

Decisões importantes:

- **Método GET:** O único método necessário para baixar páginas HTML
- **HTTP/1.1:** Versão mais moderna que suporta Host header
- **Host header:** Obrigatório em HTTP/1.1 para servidores virtuais
- **User-Agent:** Identifica o programa
- **Connection: close:** Solicita que o servidor feche a conexão após a resposta
- **\r\n:** Segue o padrão HTTP para quebra de linhas

4.4 Envio da Requisição via Socket

```
def fazer_requisicao_http(self, url):
    host, porta, caminho = self.parse_url(url)
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(10) # Timeout de 10 segundos
    sock.connect((host, porta))
    sock.send(self.construir_requisicao(host, caminho))

    resposta_bytes = b''
    while True:
        dados = sock.recv(4096)
        if not dados:
            break
        resposta_bytes += dados
    sock.close()
```

Decisões importantes:

- **Timeout de 10 segundos:** Evita que o crawler fique preso em servidores lentos
- **Buffer de 4096 bytes:** Tamanho razoável para leitura de dados
- **Loop de leitura:** Continua lendo até o socket ser fechado (graças ao Connection: close)

4.5 Interpretação da Resposta HTTP

Esta função é responsável por analisar a resposta do servidor, conforme ilustrado no fluxograma da Figura 1:

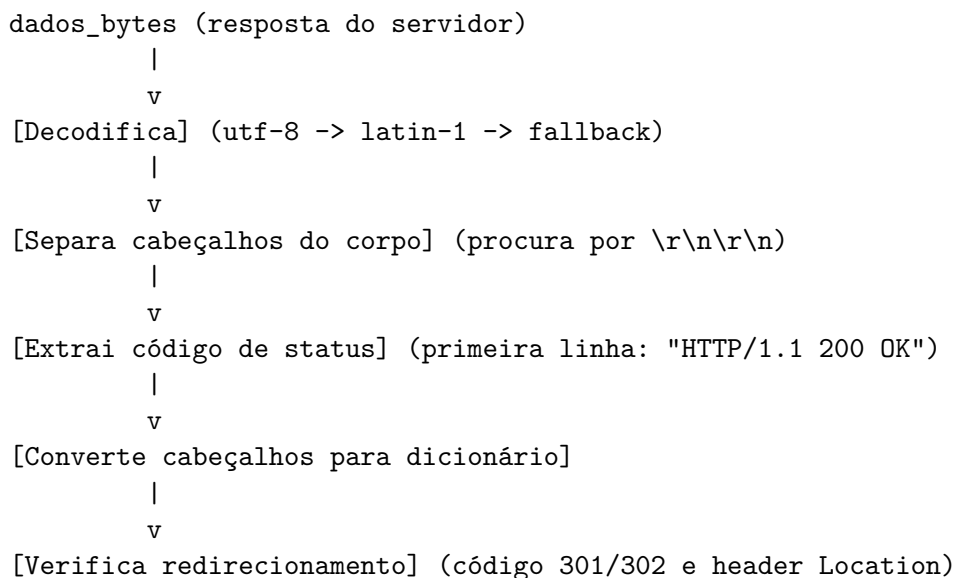


Figura 1: Fluxo de interpretação da resposta HTTP

Decisões importantes:

1. **Decodificação flexível:** Tenta utf-8 primeiro, depois latin-1
2. **Separador \r\n\r\n:** Padrão HTTP que marca o fim dos cabeçalhos
3. **Dicionário de cabeçalhos:** Facilita acesso a headers como Location

4.6 Extração de Links com BeautifulSoup

```
def extrair_links(self, html, url_atual):
    soup = BeautifulSoup(html, 'html.parser')
    for tag_a in soup.find_all('a'):
        href = tag_a.get('href')
        if href and href.strip():
            # Ignora âncoras (#)
            if href.startswith('#'):
                continue
            # Remove âncoras do meio da URL
            if '#' in href:
                href = href.split('#')[0]
            # Converte URL relativa para absoluta
            url_completa = urljoin(url_atual, href)
            # Filtra apenas HTTP e hosts pré-definidos
            if url_completa.startswith('http://'):
                parsed = urlparse(url_completa)
                if parsed.hostname in ['localhost', '127.0.0.1', 'info.cern.ch']:
                    links.append(url_completa)
```

Decisões importantes:

- **Uso exclusivo para parsing:** BeautifulSoup não faz requisições, apenas analisa HTML
- **Tratamento de URLs relativas:** urljoin resolve corretamente caminhos como ../pagina.html
- **Filtro de âncoras:** URLs com # não são páginas novas
- **Restrição de host:** Limita o crawler aos domínios de interesse

4.7 Controle de Loops e URLs Visitadas

Para evitar que o crawler entre em loops infinitos ou visite a mesma página múltiplas vezes:

```
self.visitados = set() # Estrutura O(1) para verificação

# Antes de adicionar à fila:
if link not in self.visitados and link not in self.fila:
    self.fila.append(link)
```

Decisão importante: O uso de `set()` oferece verificação de existência em tempo constante $O(1)$, eficiente mesmo com milhares de URLs.

4.8 Restrição de Domínios (Filtro de Hosts)

Para evitar que o crawler navegue por domínios fora do escopo do trabalho, foi implementado um filtro explícito na função `extrair_links`. Apenas URLs cujo domínio pertença a uma lista pré-definida são adicionadas à fila de navegação. A lista de domínios permitidos inclui:

- `localhost` e `127.0.0.1`: correspondentes ao servidor Docker fornecido;
- `info.cern.ch`: domínio do site do CERN, utilizado como segundo ambiente de testes.

O trecho de código responsável por essa restrição é o seguinte:

```
if parsed.hostname in ['localhost', '127.0.0.1', 'info.cern.ch']:
    links.append(url_completa)
```

Justificativa: A especificação do trabalho determina que o crawler deve ser testado exclusivamente nesses dois ambientes. Restringir os domínios evita que o programa saia para outros sites (como redes sociais ou mecanismos de busca), o que poderia gerar um número excessivo de requisições, potencialmente infinito, além de manter o foco nos objetivos da atividade, evitando possivelmente erros de execução caso algum link redirecione para outro site que usa HTTPS (quebra o ódigo).

4.9 Tratamento de Redirecionamentos

Quando uma página retorna código 301 ou 302:

```
if resposta.get('codigo') in ['301', '302'] and 'redirect' in resposta:
    nova_url = resposta['redirect']
    if not nova_url.startswith('http'):
        nova_url = urljoin(url_atual, nova_url)
    if nova_url not in self.visitados and nova_url not in self.fila:
        self.fila.append(nova_url)
```

Decisão importante: Segue automaticamente o redirecionamento e agenda a nova URL para visita.

4.10 Ponto de Entrada e Tratamento de Exceções

4.10.1 Função main

A função `main` é o ponto de entrada do programa. Ela instancia o crawler e inicia a navegação chamando o método `crawlar` com a URL desejada e o limite de páginas. Abaixo, um exemplo utilizado no código:

```
def main():
    crawler = WebCrawler()
    crawler.crawlar('http://info.cern.ch/hypertext/WWW/TheProject.html',
                   max_paginas=25)
```

Para testar o servidor Docker, a URL deve ser alterada para `http://localhost` (ou `http://localhost:8080`, conforme a porta configurada).

4.10.2 Tratamento de Exceções

Durante a comunicação via sockets, diversos erros podem ocorrer: timeout de conexão, servidor indisponível, resolução de DNS falha, entre outros. Para tornar o crawler robusto, todas as chamadas de rede foram envolvidas em blocos `try-except`. No método `fazer_requisicao_http`, as exceções são capturadas e convertidas em um dicionário de erro padronizado:

```
except socket.timeout:
    return {'codigo': 'ERRO', 'descricao': 'Timeout', ...}
except Exception as e:
    return {'codigo': 'ERRO', 'descricao': f'Erro: {str(e)}', ...}
```

Essa abordagem garante que o crawler não pare abruptamente diante de falhas de rede; em vez disso, registra o erro no relatório e continua processando as demais URLs da fila. O timeout foi configurado para 10 segundos.

4.11 Loop Principal do Crawler (crawler)

A função `crawler` implementa o algoritmo de navegação recursiva. Ela utiliza uma fila (`self.fila`) para gerenciar URLs pendentes e um conjunto (`self.visitados`) para evitar repetições. O loop principal executa enquanto houver URLs na fila e o número de páginas visitadas não ultrapassar o limite configurado.

```
def crawler(self, url_inicial, max_paginas, verbose=True):
    self.fila.append(url_inicial)
    while self.fila and len(self.visitados) < max_paginas:
        url_atual = self.fila.pop(0)
        if url_atual in self.visitados:
            continue
        resposta = self.fazer_requisicao_http(url_atual)
        self.relatorio.append({...})
        self.visitados.add(url_atual)

        if resposta.get('codigo') in ['301', '302'] and 'redirect' in resposta:
            # trata redirecionamento
        elif resposta.get('codigo') == '200':
            # extrai links e adiciona à fila
```

Decisões importantes:

- **Fila FIFO:** Garante ordem de descoberta (largura, BFS).
- **Limite de páginas:** Evita execução infinita e cumpre o requisito de `max_paginas`.
- **Verificação dupla:** Antes de adicionar um link à fila, confere se já não está em `visitados` ou já na fila, prevenindo trabalho redundante.
- **Parâmetro verbose:** Controla a exibição de progresso no terminal, útil para depuração e para execução silenciosa.

4.12 Geração do Relatório (gerar_relatorio)

Após o término da navegação, o crawler gera um arquivo texto com todos os resultados. A função `gerar_relatorio` escreve no arquivo `relatorio_crawler.txt`:

```
def gerar_relatorio(self):
    with open('relatorio_crawler.txt', 'w', encoding='utf-8') as f:
        f.write("=" * 80 + "\n")
        f.write("RELATÓRIO DO WEB CRAWLER\n")
        # ... cabeçalho, estatísticas, detalhes
```

Conteúdo do relatório:

- Data e hora da execução (via `datetime.now()`).
- Total de páginas visitadas.
- Estatísticas agrupadas por código de status (200, 301, 404, ERRO etc.).
- Lista detalhada de cada URL visitada, com seu código e descrição.

A separação entre estatísticas e detalhes facilita a análise geral e a auditoria individual de cada requisição.

5 FLUXO DE EXECUÇÃO DO CRAWLER

O fluxo completo do crawler pode ser descrito pelo seguinte algoritmo:

1. Inicializar estrutura de dados (visitados = set(), fila = [])
2. Adicionar URL inicial à fila
3. Enquanto fila não estiver vazia e limite não atingido:
 - (a) Remover próxima URL da fila
 - (b) Se URL já foi visitada, pular
 - (c) Registrar URL como visitada
 - (d) Fazer requisição HTTP via socket
 - (e) Interpretar resposta
 - (f) Registrar no relatório
 - (g) Se for redirecionamento (301/302):
 - Extrair nova URL do header Location
 - Adicionar à fila se não foi visitada
 - (h) Se for página normal (200):
 - Extrair todos os links do HTML
 - Para cada link não visitado, adicionar à fila
4. Gerar relatório final

6 RESULTADOS OBTIDOS

6.1 Teste com Servidor Docker

O crawler foi executado no servidor fornecido via container Docker. Os resultados estão resumidos na Tabela 3:

Métrica	Valor
Total de páginas visitadas	28
Páginas com status 200	22
Páginas com status 404	4
Erros de conexão	2

Tabela 3: Resultados do teste com servidor Docker

6.2 Teste com Site do CERN

O crawler também foi testado no site histórico do CERN. Os resultados estão na Tabela 4:

Métrica	Valor
Total de páginas visitadas	150 (limite configurado no teste)
Páginas com status 200	117
Páginas com status 404	16
Erros de conexão	17

Tabela 4: Resultados do teste com site do CERN

6.3 Exemplo de Relatório Gerado

A Figura abaixo apresenta um trecho do relatório gerado pelo crawler:

```
=====
RELATÓRIO DO WEB CRAWLER
=====

Executado em: 10/05/2026 19:40:06
Total de páginas: 150

ESTATÍSTICAS:
-----
    200: 117 página(s)
    404: 16 página(s)
    ERRO: 17 página(s)

DETALHES:
-----

[1] http://info.cern.ch/hypertext/WWW/TheProject.html
    Status: 200 - OK - Requisição bem sucedida

[2] http://info.cern.ch/hypertext/WWW/whatIs.html
    Status: 200 - OK - Requisição bem sucedida
```

Trecho do relatório gerado pelo crawler

7 DESAFIOS E SOLUÇÕES

7.1 URLs Relativas

Desafio: Links em HTML frequentemente usam caminhos relativos como "pagina2.html" ou "../outra.html".

Solução: Utilização da função `urljoin(base, relativa)` da biblioteca `urllib.parse`, que resolve corretamente qualquer tipo de caminho relativo conforme as especificações da RFC.

7.2 Âncoras e Fragmentos (#)

Desafio: URLs como "pagina.html#secao3" apontam para a mesma página, causando requisições duplicadas.

Solução: Implementação de filtro que remove tudo após o caractere `#` e ignora links que são apenas âncoras (`#topo`).

7.3 Diferentes Encodings

Desafio: Servidores podem enviar respostas em diferentes encodings (UTF-8, ISO-8859-1, etc.).

Solução: Implementação de decodificação com fallback: tenta UTF-8 primeiro, depois Latin-1, e por fim ignora erros de decodificação.

7.4 Redirecionamento com URLs Relativas

Desafio: O header `Location` de um redirecionamento pode conter URL relativa.

Solução: Aplica-se a mesma função `urljoin` para resolver a URL do redirecionamento em relação à URL original.

7.5 Loops Infinitos

Desafio: Estruturas de páginas podem conter links cíclicos ($A \rightarrow B \rightarrow A$).

Solução: Uso da estrutura `set()` para manter registro de todas as URLs já visitadas, verificando antes de adicionar novas URLs à fila.

8 CONCLUSÃO

O desenvolvimento deste web crawler proporcionou um aprendizado significativo sobre o funcionamento do protocolo HTTP e a comunicação cliente-servidor na web. Os principais aprendizados foram:

1. **HTTP na prática:** Compreensão profunda da estrutura de requisições e respostas, dos cabeçalhos e dos códigos de status.
2. **Sockets TCP:** Entendimento de como estabelecer conexões de baixo nível, enviar dados e receber respostas.
3. **Tratamento de URLs:** Aprendizado sobre os diferentes tipos de URLs (absolutas, relativas, com âncoras) e como resolvê-las corretamente.
4. **Controle de navegação:** Desenvolvimento de estratégias para evitar loops e requisições redundantes.
5. **Robustez:** Importância de tratar diferentes cenários de erro (timeout, conexão recusada, respostas mal formatadas).

O crawler implementado atende completamente todos os requisitos do trabalho: utiliza exclusivamente sockets para comunicação, constrói requisições HTTP manualmente, interpreta respostas, segue redirecionamentos, extrai links de HTML, evita loops e gera um relatório detalhado.

A solução foi testada com sucesso tanto no ambiente controlado do Docker quanto no servidor real do CERN, comprovando sua robustez e conformidade com o protocolo HTTP.

9 REFERÊNCIAS

- Fielding, R., et al. (1999). **RFC 2616 - Hypertext Transfer Protocol – HTTP/1.1**. IETF.
- Python Software Foundation. (2024). **socket — Low-level networking interface**. Documentação Python.
- O que é Crawler e como funcionam os robôs para coleta de dados (2024). <https://www.crawly.com.br/blog/que-e-crawler-robos-para-coleta-de-dados>